# Analysing Textual Reviews of Kebab Restaurants in Krakow and Warsaw

Wiktor Prosowicz[1], Jan Stryszewski[2], and Oğuzhan Berke Özdil[3]

[1]AGH University of Krakow; wprosowicz@student.agh.edu.pl
[2]AGH University of Krakow; yanco@student.agh.edu.pl
[3]AGH University of Krakow; ozdil@student.agh.edu.pl

## ABSTRACT

In this paper, we present our analysis of textual reviews of kebab restaurants, scraped from Google Maps. We aimed to perform thorough experiments involving setting up the scraping pipeline, preprocessing the data, extracting useful numerical representations and training neural network-based models, recognizing the sentiment of the reviews. Each review was converted into a set of linguistic representations, from classical ones (e.g. TF-IDF) to modern Transformer-based embeddings, and numerical features. We subsequently trained multiple configurations of neural network-based models to predict review rating on scale 1 to 5. Obtaining the results allowed us to gain meaningful insight into the data and draw conclusions on the applicability of different NLP techniques for sentiment analysis.

Keywords: natural language processing, neural networks, sentiment analysis

## 1 INTRODUCTION

We consider the following situation: Yusuf Caner, a Turkish cook, once learned that kebabs are extremely popular in central and eastern Europe and decided to relocate his business to Poland. Now he runs two restaurants: one in Krakow and one in Warsaw. Each day, his restaurants receive numerous textual reviews that clients place on their website. Alas, the number of them has grown so huge that Mr Caner is not able to read them all, yet he wants to know more or less what his clients think about the quality of service. In particular, he wants to estimate the overall rating of his restaurant and be able to take a look at the worst reviews, containing useful advices and criticism.

He hired us, a certified team of data scientists, to build a tool specialized in predicting the sentiment of textual reviews. The following applies in this situation:

1. The tool has to take a textual review as input and output the sentiment score, either in fine scale 1 to 5 or coarse scale 1 to 3 (negative, neutral, positive).

2. Mr. Caner is able to provide us with information about the particular restaurant mentioned in each review (Krakow or Warsaw).

### 1.1 Problem formulation

We try to build a model that predicts the rating of a restaurant based on its review. To this end, we scraped a certain number of labeled reviews from Google Maps and explored the possible ways of processing them to achieve the most meaningful representation of a single review, which would be subsequently used to estimate its sentiment with the highest accuracy possible.

To ensure reproducibility, we share the repository containing code and data used to obtain the result presented in this paper: https://github.com/WiktorProsowicz/advanced-data-mining-project.

## 2 LITERATURE REVIEW

The methods and materials used in this work are in most cases standard NLP techniques, mentioned in numerous works, either academic papers or textbooks. Regarding the classical forms of text representation, we base our understanding of them on chapters 3 and 5 of the book *Deep Learning for NLP and Speech Recognition* (1). We gained our knowledge about the construction of advanced contextual text embeddings

either from the original BERT paper (2) or works incorporating advanced context modeling into TTS pipelines (3; 4).

## 3 MATERIALS AND METHODS

In this section, we describe all phases of the development of our project. Specifically, we sketch out the data scraping pipeline and configuration, the processing of the data, and the neural architecture of the rating prediction model.

### 3.1 Obtaining the data

In order to scrape the Google Maps reviews from a chosen set of restaurants, we need an automated pipeline that is able to interact with the service in a similar way a human would. This is crucial since only this way we can leverage the internal Google's algorithms for restaurant lookup, as well as loading, sorting, and automated translation of the reviews. To this end, we incorporated Microsoft's package Playwright [1], which allows us to automatically interact with a browser engine.

The algorithm is presented in 1.

**Algorithm 1** Scraping raw reviews info from Google Maps

**Require:** $Q$ - a set of restaurant lookup queries (e.g. "krakow old town kebab"), $n_restaurants$ - maximum restaurants to take, $n_reviews$ - maximum reviews to take for a single restaurant

```
Init RawDS - empty dictionary mapping grouping reviews by restaurant
Send an HTTP get request to the main Google Maps page
for q in Q do
    Init R - empty list of structures describing restaurants
    Type q in the search bar and trigger the search operation
    Navigate to the scrollable bar with restaurants
    for i up to min(n_restaurants, num available restaurants) do
        Append info about i^th restaurant to R
    end for
    for r in R do
        Send an HTTP get request to the Google Maps site of the restaurant
        Navigate to the reviews bar
        for i up to min(n_reviews, num available reviews) do
            Append info about the i^th review to RawDS(r)
        end for
    end for
end for
return RawDS
```

### 3.2 Raw dataset

The data scraping algorithm, described in section 3.1, yields a dictionary grouping reviews by the restaurant to which they refer. Since Google Maps automatically translates the reviews to English, we are able to obtain both language versions for the non-english reviews, and thereby determine whether the review was translated or not. The information scraped for each review includes the following field:

- **text** - contains the English content of the review.
- **rating** - an integer number from 1 to 5.
- **translated** - boolean label that tells if the review has been translated by the Google Maps engine.
- **original** - if **translated** is true, the field contains the original version of the review.

An example review is presented in the listing 1.

[1] https://github.com/microsoft/playwright

**Listing 1** Raw information corresponding to an example review.

```
{
    "text": "Kebab very average and almost cold. The seller
        must be hard of hearing because you order it with a
        mild sauce and get it with a spicy one. I don't
        recommend it.",
    "rating": 1.0,
    "translated": true,
    "original": "Kebab bardzo średni i prawie zimny. Sprzedawca
        chyba niedoslyszacy bo zamawiasz z sosem  lagodnym a
        dostajesz z  ostrym. Nie polecam."
}
```

### 3.3 Data processing

The data processing pipeline was designed to extract a set of numerical representations from each review. The idea was to treat each review as a single data sample; therefore, the restaurant's location information was inherited by its reviews.

The first step of the pipeline was text normalization, conducted similarly as in the preprocessing pipelines of numerous Text-to-Speech systems, i.e. involving transformations, such as converting numbers, dates and currency symbols into their alphabetical forms, removing emoji symbols, etc. To perform the aforementioned processing, we used the *gruut* package [2], specifically the text normalization part of its grapheme-to-phoneme pipeline.

The following subsections describe certain groups of numerical features prepared for each review.

#### *3.3.1 Linguistic vectors*

The normalized text of each review was used to prepare a vocabulary that contains all unique words, except English stop words, such as *"the"*. The algorithm kept a count of additional information, such as the total occurrences of each word and the number of unique documents that contain it. The vocabulary was subsequently used to create three types of vectors: 1) **Top** - contains N most frequent words; 2) **Bottom** - contains M least frequent words; 3) **Full** - contains all words. Each vocabulary group was used to produce TF-IDF and Binary Bag-of-Words representations.

Moreover, we leveraged a pre-trained BERT model to create context-aware dense sentence representations. For each sentence in the review, we obtain the embedding vector corresponding to the *[CLS]* token. Then, all such vectors were averaged to compose a single review representation. We used a publicly available model form the Hugging Face library, called *bert-large-cased*.

The last linguistic feature we consider in this research are Part-of-Speech (POS) tags. These are assigned to each word and refer to the part of speech it represents in the analyzed sentence. POS tags are especially useful in cases where we want to disambiguate between different meanings of two instances of a certain word that do not differ in their syntactic form, e.g., the word *read* in present and past tense. We used POS tags to obtain the overall description of the text, that is, we represent the review with a bag-of-words vector containing counts of the POS tags found there.

#### *3.3.2 Numerical features*

Besides the standard vectorized representations of text, we decided to obtain simple numerical statistics, depicting the overall characteristics of each review's content. These are specifically the number of words, the number of sentences and the boolean label that refers to the location of the restaurant.

The rationale behind using such simple features was as follows: we believe that there might be some relationship between, for example, the overall "shape" of the sentences and the sentiment of the review. For example, especially angry customers from Warsaw might use multiple short sentences to criticize multiple flaws of the service. Also, there might be hidden relationships between the linguistic features and the location of the restaurant: we can imagine that the people living in a certain town (which probably

[2] https://github.com/rhasspy/gruut

make up the majority of the clients) use a given word to describe their positive feelings with respect to the food, which in other places might be used differently.

#### 3.3.3 Trace features

Another class of numerical representations comprises the features we call *trace features*, inspired by (5). The work describes a technique of representing the core semantic characteristic of a text with two numbers, called *speed* (referred to by us from this point onward as *velocity*) and *volume*.

This representation is constructed as follows: first, all words in the text are replaced with their dense vectorized representations through either a static corpus of word embeddings or a context-aware technique (in this work, we use token-level representations obtained from the BERT model, described in section 3.3.1). Then, the embeddings are averaged within specified chunks producing points in the embedding space, which represent overall topics mentioned in the text. In this way, each text is a trace which may be described by the average speed of jumping from topic to topic and the volume spanned by it, being the measure of the general richness of the analyzed text. We calculated the chunk representations using a sliding window with a specified length and step size, similarly as in the Short-Time Fourier Transform.

Let us denote the trace representation as

$$Trace(Review) = (c_i \in \mathbb{R}^D \; for \; i \; in \; 1..N) \tag{1}$$

where $N$ is the number of chunks and $D$ is the dimensionality of the embedding vectors. We calculate the features described above as follows: the velocity is the average distance between each pair of neighboring points in the trace:

$$Velocity(Trace) = mean(\|c_i - c_{i+1}\|_2 \; for \; i \; in \; 1..N-1) \tag{2}$$

The volume, in turn, was calculated as the euclidean norm of a vector containing the distances between minimum and maximum values for each dimension of the chunk representations:

$$\begin{aligned} SpanVec(Trace) &= [max(c_1[i], ..., c_N[i]) - min(c_1[i], ..., c_N[i]) \; for \; i \; in \; 1..D] \\ Volume(Trace) &= \|SpanVec(Trace)\|_2 \end{aligned} \tag{3}$$

### 3.4 Neural architecture

The complete neural architecture of the system is presented in Figure 1. It depicts the flow of the data considering various classes of input features. Each final model is characterized by the subset that contains particular input features that it processes. The overall processing pipeline is as follows:

1. Each linguistic vector is processed by a separate *Linguistic Encoder*.
2. The chosen numerical and trace features are concatenated and processed by the *Numerical Features Encoder*.
3. The output vectors of all linguistic encoders and the numerical encoder are concatenated and processed through the *Post Net*.
4. The post net output is used to create two types of output: the primary 5-class classification output and the secondary regression output.

The following subsections describe particular elements of the architecture in detail.

#### 3.4.1 Linguistc Encoder

The role of linguistic encoders is to squash the (possibly sparse) linguistic features into a low-dimensional dense vector. Each encoder contains several blocks which gradually lower the dimensionality of the processed vector. Each block consists of a fully-connected layer with ReLU activation, followed by batch normalization and dropout.

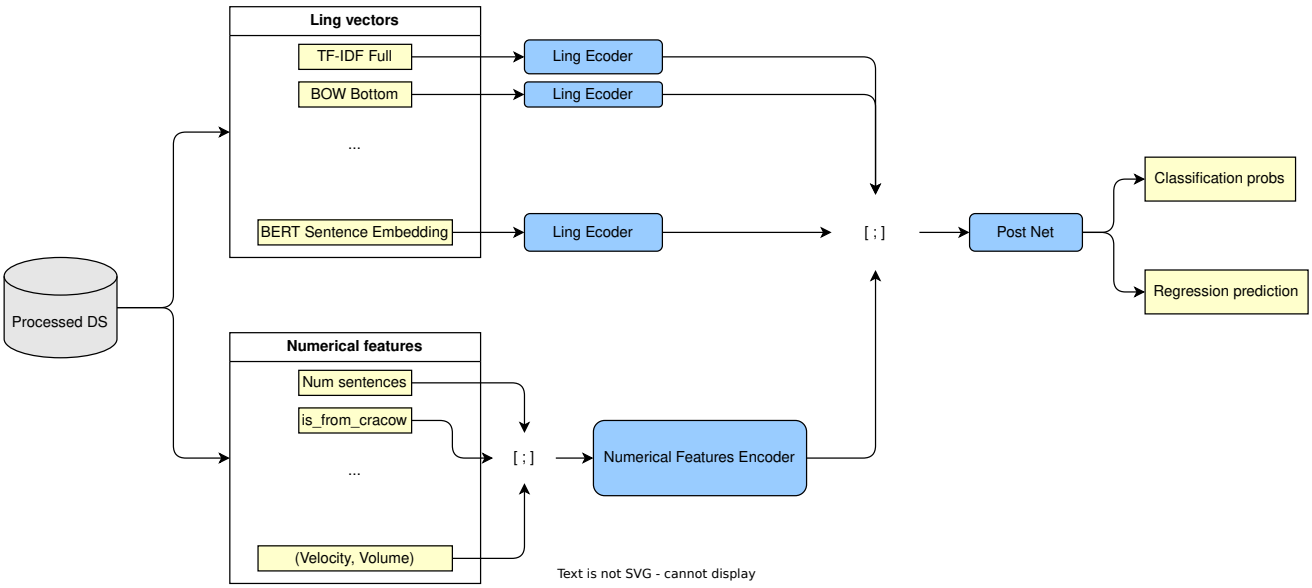



**Figure 1.** The overview of the proposed system's neural architecture.

#### *3.4.2 Numerical Features Encoder*

The numerical features encoder processes the concatenated numerical features and squashes them into a lower dimensionality representation. Since we believe that the relationships between the numerical features might be more complex than those present in linguistic vectors, we decided to use a slightly different architecture. Specifically, we replaced the processing blocks with a stack of Kolmogorov-Arnold Networks, which have been proven to exhibit better modeling capabilities than the classical MLP layers. The layer stack's output is processed through a batch normalization layer and dropout regularization.

#### *3.4.3 Post Net*

The post net processes the encoded linguistic and numerical features jointly and outputs the rating prediction, with respect to which the loss function is calculated. The primary output is a five-dimensional vector with softmax activation, which predicts the rating class on the scale 1 to 5. The secondary output is a single number that indicates the predicted rating in the continuous domain. The rationale behind using twofold output is that we this way we can give the model some indication of the distance between classes (confusing 1 with 5 is simply less desired than confusing 1 with 2).

The architecture of the post net consists of a stack of processing blocks, described in section 3.4.1.

### 3.5 Training

The model is trained to minimize the weighted loss:

$$\mathscr{L} = \lambda_1 \mathscr{L}_{classification} + \lambda_2 \mathscr{L}_{regression} \tag{4}$$

where $\mathscr{L}_{classification}$ is the categorical cross-entropy between the predicted class probabilities and the true class labels. $\mathscr{L}_{regression}$ is the mean square error between the predicted and target ratings. To achieve the best test accuracy possible, we leveraged a technique called Stochastic Weight Averaging, which calculates the average of the model's weights in the last training phase to ensure the model does not end up in a sharp minimum, but rather stops in the center of the flattened loss function surface and consequently generalizes to the test set better.

#### *3.5.1 Metrics*

In order to evaluate the model's performance both during the training and testing phase, we determined a set of metrics, calculated with respect to the model's output. The primary metrics refer to the classification output and these are namely the accuracy, recall and F1 score. Some metrics mentioned above are split into four types by the following pivot properties:

- **coarse/fine** - specifies whether the metric refers to general 3-class or full classification. The original predicted classes are converted to the coarse scale by the following mapping: $(1,2) \rightarrow 1,\ (3) \rightarrow 2,\ (4,5) \rightarrow 3$.
- **weighted/macro** - determines how the in-class metric values are averaged. In the *weighted* metrics, the values are first scaled with respect to the recorded counts of the classes so that the most common ones are prioritized. In the *macro* setup, the metric is calculated as the arithmetic mean of the values.

In order to measure the accuracy of the regression output, we calculate the mean absolute error (MAE) between the output and the target values.

## 4 EXPERIMENTS & RESULTS

This section describes the details related to the experiments we conducted to evaluate our method. We present the setup of the data scraping pipeline, training configurations, and the results obtained from the experiments.

### 4.1 Dataset

After scraping a considerable number of reviews, we performed a basic exploratory data analysis to obtain insight into the underlying data distributions. Table 1 presents the basic statistics calculated for the obtained raw dataset. The distributions of review ratings and basic numerical features are presented in Figures 2 (a) and (b), respectively.

**Table 1.** Overall statistics describing the scraped raw dataset.

| Property | Value |
|---|---|
| Total restaurants | 253 |
| Total reviews | 61956 |
| (Min, Max, Avg) restaurant rating | (2.27, 5.0, 4.05) |
| (Min, Max, Avg) words per review | (1, 1012, 40.7) |
| (Min, Max, Avg) sentences per review | (1, 59, 3) |
| Number of reviews from Krakow | 29993 |
| Number of translated (non-english) reviews | 47654 |

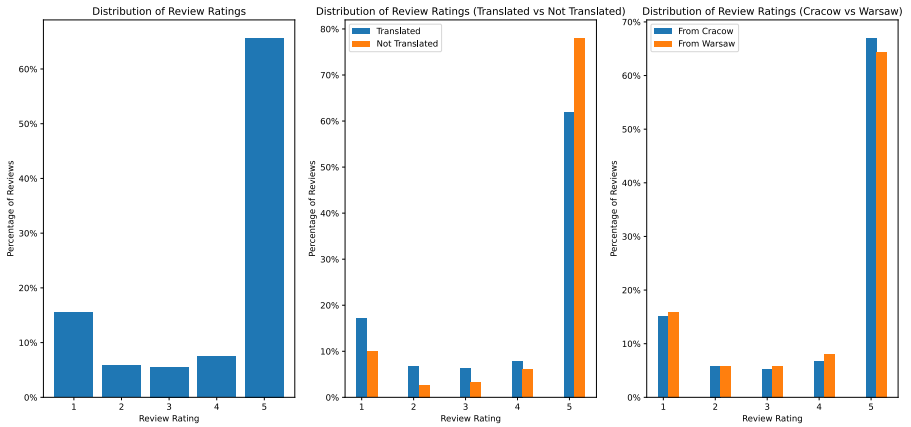
While processing the raw data, we constructed a vocabulary set with around 20K distinct words. Among the most frequent words, there are the most intuitive ones, referring to the personal feelings about the service (*good*, *delicious*, *fresh*, etc.) and description of the particular elements of the served food (*meat*, *sauce*, *large*, etc.). As expected, the least frequent words contain those that were improperly translated, named entities or mispronounced words (*kebabitos*, *służewiec*, *opsuga*, *kurwaaaa*, *lavaaaaaaaasz*, etc.).

We decided to build the *top* and *bottom* linguistic representations using 2K most frequent and 4K least frequent words, respectively. The reason for this dissimilarity is that we expect the latter vocabulary set to contain a lot of extremely uncommon words, which may make it harder for the model to learn to predict the correct rating. This fact is reflected in the initial analysis of the processed data set, namely in Figure 3. It is clear that: 1. The *bottom* representations contain a considerable number of all-zero vectors; 2. The average number of non-zero elements is low.

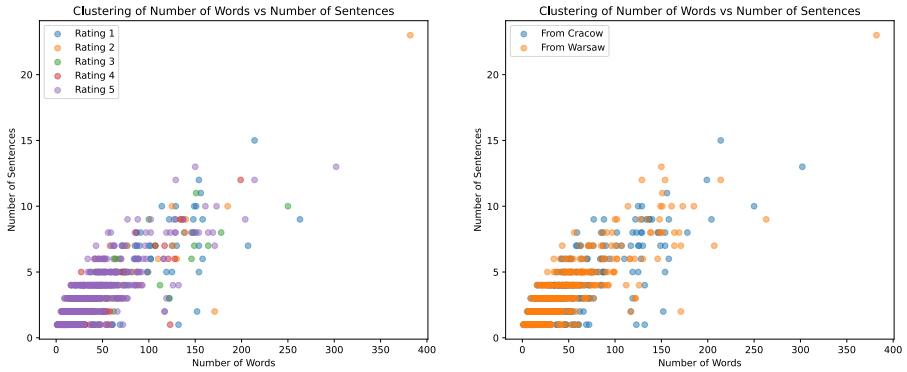
We collected the trace features using the following chunk sizes and step lengths:

- Chunk length = 3, step size = 1
- Chunk length = 3, step size = 2
- Chunk length = 5, step size = 2
- Chunk length = 5, step size = 3
- Chunk length = 7, step size = 3

The (velocity, volume) pairs are visualized in Figure 4. Note that the data points are clustered in a triangular shape, which means that the higher the volume, the closer to the average the velocity.




**(a)**




**(b)**

**Figure 2.** Figures presenting the underlying distributions of the raw data. **(a)** The distributions of the review ratings. From top to bottom, the figures present the overall distribution, the distributions with respect to whether the reviews were translated and the distributions with respect to the location of the restaurant. **(b** The join distribution of word and sentence counts with respect to the rating and restaurant's location.

## 4.2 Experimental setup

In order to evaluate our method and choose the best configuration of input features and model's parameters, we conducted two experiments: the role of the first was to get insight into the usefulness of the linguistic vectors, whereas the second experiment was aimed at determining the globally best model and the best configuration of the numerical features.

Each model was trained for no more than 15 epochs with the early stopping set to monitor the classification loss with patience equal to 3 epochs. After training, the testing was conducted using the model with the best validation accuracy, calculated with respect to 5 classes in the *weighted* setup. All models were trained using Adam optimizer on a single NVIDIA GeForce RTX 4070 Super with 12 Gigabytes of VRAM.

## 4.3 1st experiment

The first experiment generally aimed to determine the amount of information each linguistic representation can provide.

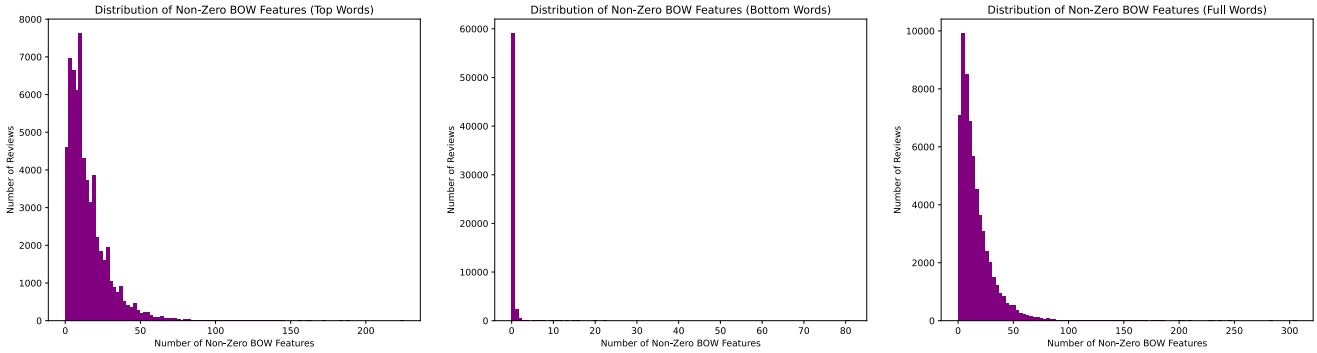



**Figure 3.** The distributions of counts of non-zero elements in binary bag-of-words representations. From top to bottom, the plots show the distibutions calculated with respect to *top*, *full*, *bottom* setups, respectively.

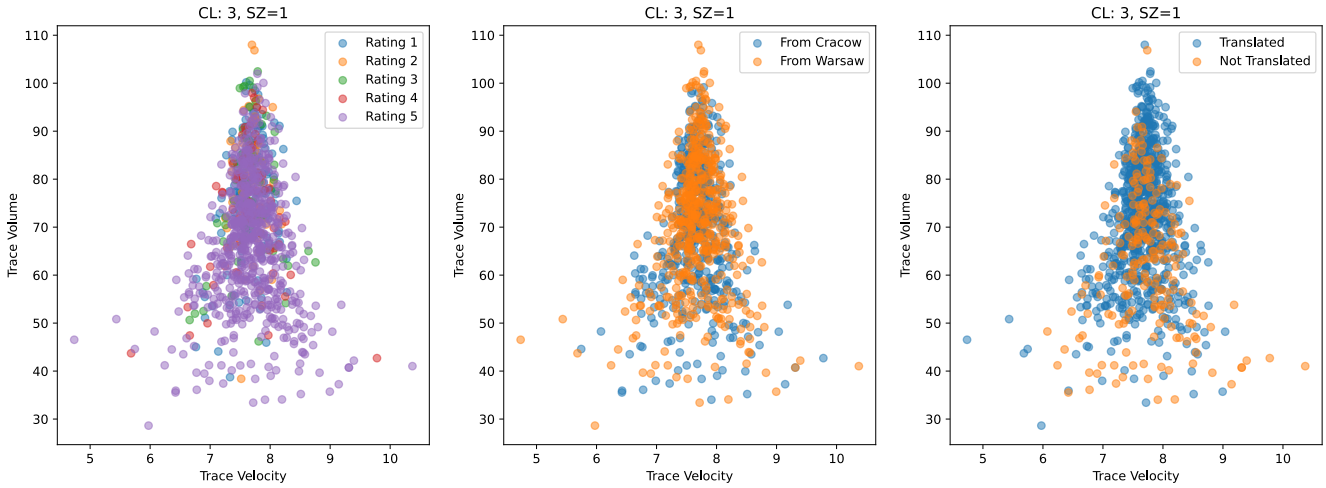



**Figure 4.** The distributions of (velocity, volume) pairs, collected for the chunk length and step size equal to 3 and 1, respectively. From left to right, the plots present the clustering of data points with respect to the rating, restaurant's location and translation label. The points with zero velocity and volume have been removed from the plots.

#### *4.3.1 Configuration*

The primary characteristic of each training configuration was the class of linguistic features it received as input (*tf-idf top*, *bbow full*, etc.). Each configuration was then split into two separate ones: one using the additional POS tags vector and the other not. The particular models also used differed in the number of hidden dimensions (128x32 or 256x64 for the primary linguistic encoder and 16x8 or 8x4 for the POS tags encoder) and the learning rate (1e-4, 5e-5, 2e-5). The hyperparameter sets above produced 128 configurations in total.

#### *4.3.2 Results*

Several of the best models from the first experiment are shown in Table 2. The list of all configurations, ordered by the accuracy calculated on the test set, contains at the top almost solely the models using *full* versions of the linguistic features. Figure 5 shows a scatter plot with (precision, recall) data points representing the experiment runs. At first glance, it can be seen that the two best models (with relatively similar precision and recall) correspond to the models using the *full* linguistic features. The models with either worst precision or recall are the ones that use the least frequent word in their vocabulary sets.

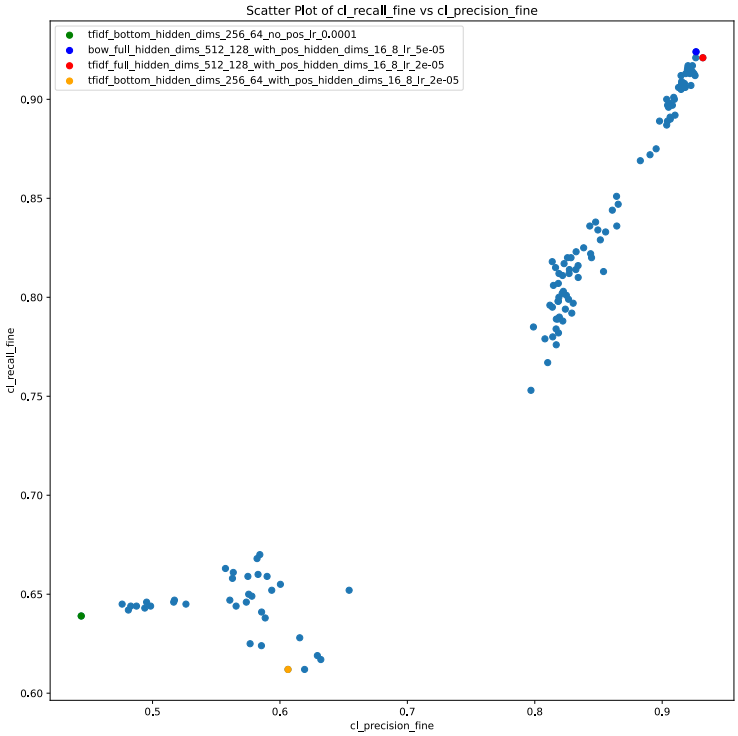



**Figure 5.** Scatter plot showing the distributions of *Recall Weighted Fine* and *Precision Weighted Fine* test metrics calculated for the runs from the first experiment.

However, since we believe that receiving the best accuracy in a single run is not enough to prove the superiority of a particular configuration of linguistic features, we decided to investigate the distributions of particular metrics with respect to certain hyperparameter configurations. Figure 6 shows the distribution of the accuracy of the prediction with respect to the linguistic features used as input. The first observation is that the *bottom* linguistic representations score the worst, whilst the configurations using *full* vectors achieved the best accuracy. It should be noted that the models using BERT sentence embeddings scored relatively low in evaluation, which may be due to insufficient modeling capabilities of the used pre-trained model. The configurations with and without POS tags encoder are grouped in pairs. An important observation is that including the POS tag vector as input does not contribute to the average performance, except for the *bottom* cases, where it improves the average accuracy by around 10 percentage points. We believe it is caused by the fact that in these cases the primary linguistic representations are simply immensely poor in information, and therefore the models may benefit at least to some minimal extent from using the additional input.

## 4.4 2nd experiment

The purpose of the second experiment was twofold: the primary goal was to find the globally best model, which included finding the best combination of linguistic features; the second purpose was to determine how the numerical and trace features contribute to the model's performance.

### *4.4.1 Configuration*

Based on our observations from the first experiment, we decided to use three primary linguistic features in the next runs: *TF-IDF Full*, *TF-IDF Top* and *Sentence BERT*. Since the *top* and *full* representation overlap, we decided not to mix them in the composed model configurations. As for other important parameters, the configurations use either a single numerical feature or all of them simultaneously and additionally a

**Table 2.** Test results collected for the top 5 best models from the 1st experiment. The rows are ordered by the *Accuracy Weighted Fine* metric.

| Ling features used | Main encoder hidden dims | POS BOW hidden dims | Learning rate | Accuracy Macro Coarse | Accuracy Macro Fine | Precision Weighted Fine | Recall Weighted Fine | Regression MAE |
|---|---|---|---|---|---|---|---|---|
| ['bow_full', 'pos_bow'] | [512, 128] | [16, 8] | 5e-05 | 0.8936 | 0.8481 | 0.9266 | 0.9240 | 0.2183 |
| ['bow_full', 'pos_bow'] | [512, 128] | [16, 8] | 0.0001 | 0.9101 | 0.8521 | 0.9263 | 0.9210 | 0.2201 |
| ['tfidf_full', 'pos_bow'] | [512, 128] | [16, 8] | 2e-05 | 0.9256 | 0.8868 | 0.9319 | 0.9210 | 1.1845 |
| ['tfidf_full', 'pos_bow'] | [256, 64] | [8, 4] | 5e-05 | 0.8934 | 0.8373 | 0.9203 | 0.9170 | 0.2492 |
| ['bow_full', 'pos_bow'] | [512, 128] | [16, 8] | 2e-05 | 0.9018 | 0.8580 | 0.9237 | 0.9170 | 0.6406 |

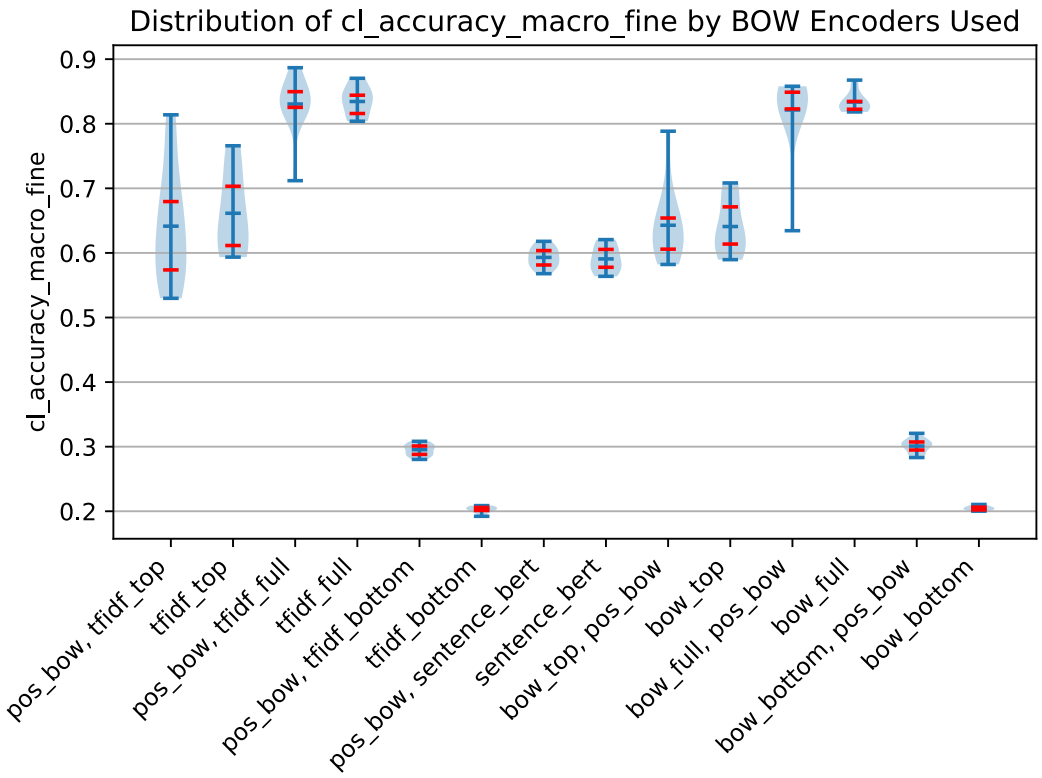



**Figure 6.** Violin plots showing the distributions of the *Accuracy Macro Fine* metric with respect to the types of input linguistic features.

chosen pair of trace features. We consider two learning rate values: 1e-4 and 5e-5. The hyperparameter sets above produced 320 configurations in total.

#### *4.4.2 Results*

Table 3 shows the configuration parameters and test metrics for the three best and three worst models, chosen with respect to the *Accuracy Weighted Fine* metric. Although the best model accepts only one linguistic feature as input, the majority of top configurations use *TF-IDF Full* and Sentence Bert linguistic features, which suggests that this configuration is the best starting point for further hyperparameter tuning. This becomes more clearly visible, once we analyze the distributions of the accuracy metric with respect to the configurations of the linguistic features, presented in Figure 7.

**Table 3.** Test results collected for the top 5 best models from the 1st experiment. The rows are ordered by the *Accuracy Weighted Fine* metric.

| Ling features used | Main encoder hidden dims | BERT encoder hidden dims | Numerical features used | Learning rate | Accuracy Macro Coarse | Accuracy Macro Fine | Precision Weighted Fine | Recall Weighted Fine | Regression MAE |
|---|---|---|---|---|---|---|---|---|---|
| ['tfidf_full'] | [512, 128] | - | ['num_sentences', 'cl_5_sz_3'] | 0.0001 | 0.8986 | 0.8671 | 0.9370 | 0.9340 | 0.2060 |
| ['tfidf_full', 'sentence_bert'] | [512, 256] | [256, 128] | ['num_sentences', 'cl_5_sz_3'] | 0.0001 | 0.9158 | 0.8885 | 0.9400 | 0.9320 | 0.2037 |
| ['tfidf_full', 'sentence_bert'] | [512, 256] | [256, 128] | ['is_from_cracow', 'cl_5_sz_3'] | 5e-05 | 0.9113 | 0.8798 | 0.9381 | 0.9310 | 0.2055 |
| ['tfidf_top', 'sentence_bert'] | [128, 64] | [128, 64] | ['num_words', 'cl_3_sz_1'] | 0.0001 | 0.7173 | 0.5822 | 0.81460 | 0.7950 | 1.8298 |
| ['tfidf_top', 'sentence_bert'] | [128, 64] | [128, 64] | ['num_words', 'cl_7_sz_3'] | 0.0001 | 0.7549 | 0.6045 | 0.8280 | 0.7950 | 0.5771 |
| ['tfidf_top', 'sentence_bert'] | [128, 64] | [128, 64] | ['is_from_cracow', 'cl_5_sz_2'] | 0.0001 | 0.7877 | 0.6200 | 0.8281 | 0.7910 | 0.5883 |

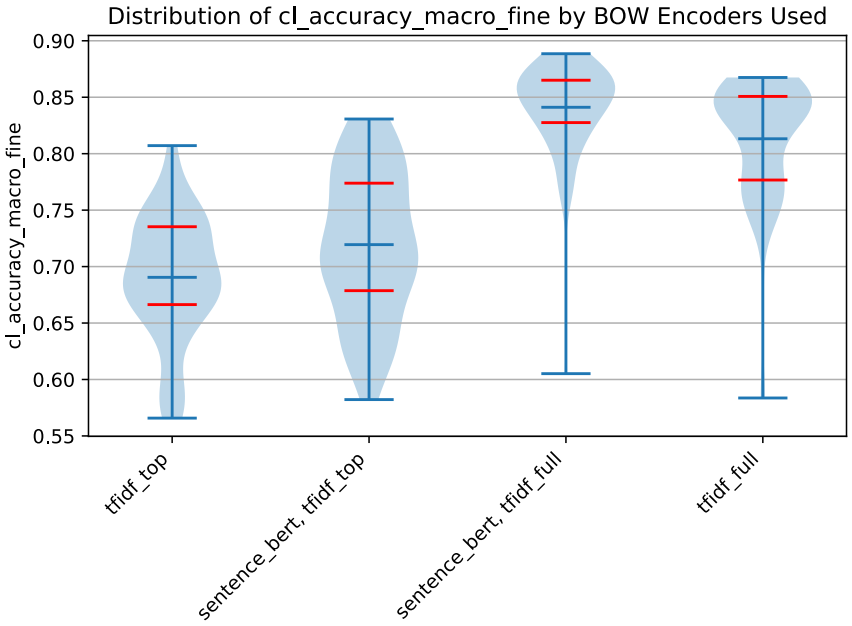
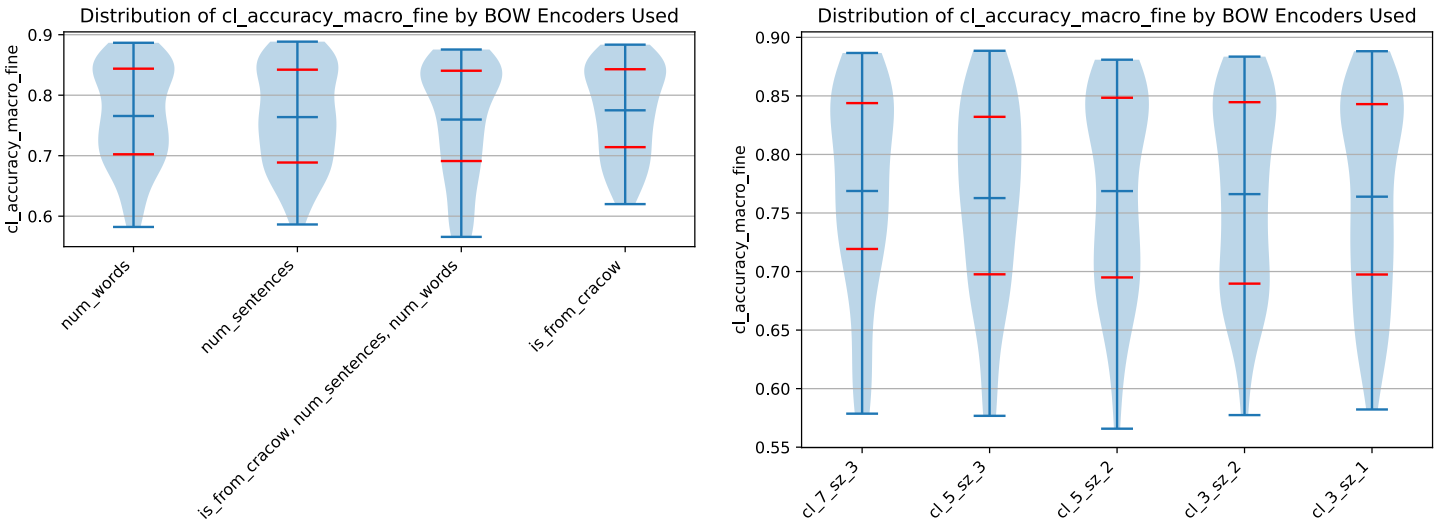



**Figure 7.** Distributions of the *Accuracy Macro Fine* metric with respect to the set of linguistic features used as the model's input.

Figures 8 (a) and (b) show a broader image of the influence of numerical and trace features on the accuracy of the prediction, respectively. Interestingly, adding either class of features does not contribute visibly to the model's performance. As for the numerical features, we believe that the problem may be either with a lack of statistically significant correlation between the features' values and the rating or lack of additional useful information provided by the features, which is not present in the used linguistic representation, just like in the case of POS tags. The poor contribution of the trace features may be attributed to the fact that they are simply ineffective in setups, where they are used to represent shorter textual content. It is also possible that they are not useful for the chosen task of sentiment analysis.




**(a)** **(b)**

**Figure 8.** Distributions of the test *Accuracy Macro Fine* metric with respect to configurations of numerical and trace features. **(a)** Distributions with respect to the numerical features. **(b)** Distributions with respect to the trace features.

## 5 DISCUSSION & CONCLUSIONS

The distributions of achieved test accuracy with respect to various classes of parameters serve as a way to assess the overall contribution of a certain parameter to the training process. Depending on what results we would like to achieve, these insights may help to find, e.g. the value of a parameter which guaranties stable training (distribution is accumulated around a reasonably high average) or makes it possible to obtain the best possible model via hyperparameter tuning.

If we treat the experiments described above as a parameter tuning procedure, we can agree on a certain choice of the *globaly best model*. In this case it is the model using *TF-IDF Full* and *Sentence BERT* linguistic representations with [512, 256] and [256, 128] linguistic encoders' hidden dimensions' configurations, respectively. The hidden dimensions used by the post net are [128, 64] and the learning rate is equal to 5e-5. The model achieved its peak accuracy in the ninth epoch of training. The rationale behind the choice lies in its evaluation scores. The model achieved the rounded values 0.94 and 0.89 of *Accuracy Macro Coarse* and *Accuracy Macro Fine* metrics, respectively.

In this paper, we present our research and development aimed at obtaining a deep understanding of possible methods to predict the sentiment of textual reviews of restaurants of a chosen type. We first presented the algorithm for scraping the data for analysis. Then, we described methods of processing the data and extracting meaningful numerical representations. We also presented a basic neural architecture that can be adjusted to a specific chosen set of input features. Through two rounds of experiments, we investigated the influence of certain classes of input features and parameter configurations on the model's performance.

## REFERENCES


[1] J. W. Uday Kamath, John Liu, *Deep Learning for NLP and Speech Recognition*. Springer Cham, 2019.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," 2019.

[3] D. Stanton, Y. Wang, and R. Skerry-Ryan, "Predicting expressive speaking style from text in end-to-end speech synthesis," 2018.

[4] Y.-J. Zhang and Z.-H. Ling, "Extracting and predicting word-level style variations for speech synthesis," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 1582–1593, 2021.

[5] J. Berger and O. Toubia, "The topography of thought," *PNAS Nexus*, vol. 3, p. pgae163, 05 2024.